

中山大学计算机院本科生实验报告

(2025 学年春季学期)

课程名称：并程序设计

批改人：

实验	MPI 并行应用	专业（方向）	计算机科学与技术
学号	22320021	姓名	陈安康
Email	1743826979@qq.com	完成日期	2025/5/17

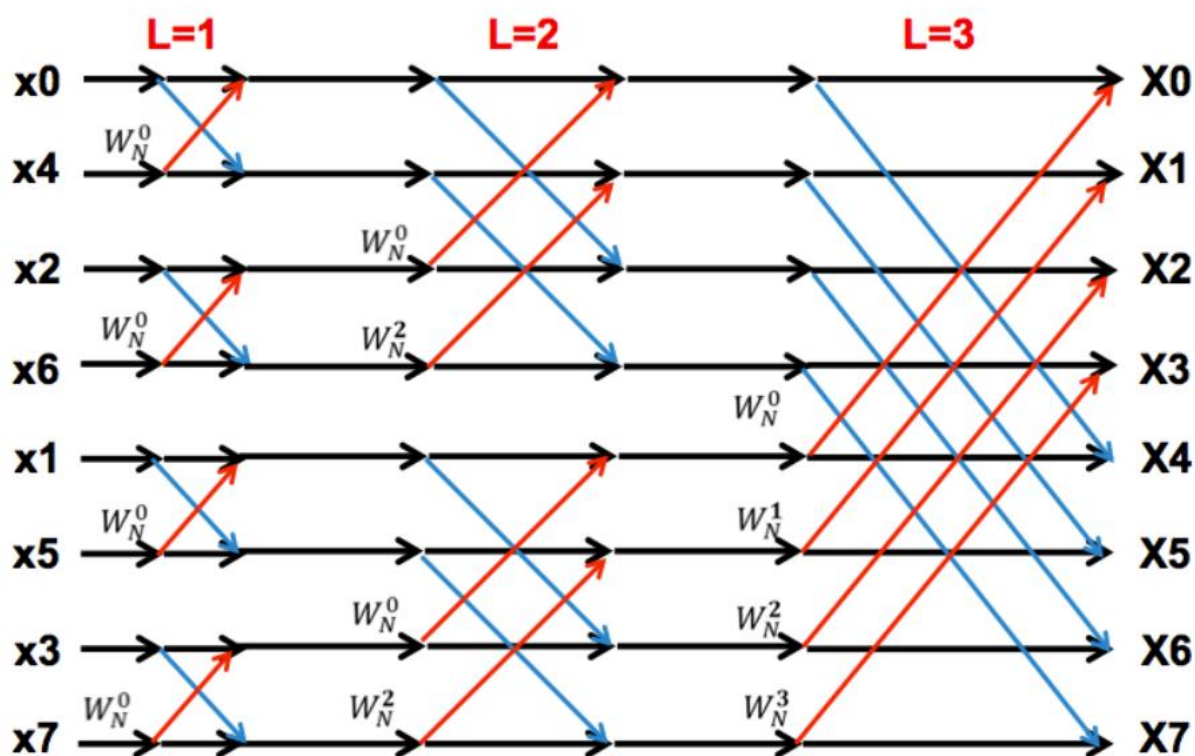
1. 实验目的

使用 MPI 对快速傅里叶变换进行并行化。对于 Lab6 实现的 `parallel_for` 版本 `heated_plate_omp` 应用进行分析。

2. 实验过程和核心代码

任务 1

根据参考资料，快速傅里叶变换的常规计算是递归完成的，流程如下图演示所示：



首先为了方便运算，最好按照图中所示进行重新的逆序排序（这种排序是将索引进行二进制翻转得到新索引），这是为了确保在后续的蝶形运算中，数

据能够正确地在不同的频率分量之间进行组合。串行化版本利用了一个规律，巧妙的使得它不必逆序排序，但这并不利于并行化的实现。并行化的思路就是将逆序排序好的数据平均分到 2 的次幂的进程中，假设进程数为 p ，则前 $\log p$ 次蝶形运算就可以完全并行，在本地进行数据计算，在大于 $\log p$ 次之后，则其余的 $\log n - \log p$ 次运算就需要进程间互相交换数据。交换规律通过上图展示也十分明显。经过运算后就将结果收集就可以得到最终的答案。

资料中还提到一种更进一步的优化方法，即在前 $\log p$ 次本地运行完成后，对数据进行转置重构，这样可以保证最后的几次运算也是在本地进行不用传递数据，不过这个实现要求数据以二维形式组织，太过复杂，这里就没有实现。

由于串行代码的代码框架不方便 MPI 的实现，所以我把代码单独拿出来实现，传入参数一次只运行一次迭代。

实现的核心代码如下：

```
void fft_parallel(double* local_data, int N, int local_N, int stages, double* W_N, int rank) {  
    for (int stage = 0; stage < stages; ++stage) {  
        int stride = 1 << stage;  
        int group_size = 2 * stride;  
  
        if (stride >= local_N) {  
            int partner = rank ^ (stride / local_N);  
            double* recv_buf = new double[2 * local_N];  
  
            MPI_Sendrecv(local_data, 2 * local_N, MPI_DOUBLE, partner, 0,  
                        recv_buf, 2 * local_N, MPI_DOUBLE, partner, 0,  
                        MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
            int flag = (partner < rank) ? -1 : 1;  
            // 蝶形运算  
            for (int k = 0; k < local_N; ++k) {  
                // 计算使用的旋转因子  
                int L = stage + 1;  
                int group_idx = rank % (stride / local_N);  
                int j = group_idx * local_N + k;  
                int index = j * (1 << (stages - L));  
                double wr = W_N[2 * (index % N)];  
                double wi = W_N[2 * (index % N) + 1];  
  
                // 保存本地数据  
                double local_real = local_data[2 * k];  
                double local_imag = local_data[2 * k + 1];  
                // 保存接收数据  
                double recv_real = recv_buf[2 * k];  
                double recv_imag = recv_buf[2 * k + 1];  
            }  
        }  
    }  
}
```

```

// 计算旋转因子与接收数据的乘积
double tr = wr * recv_real - wi * recv_imag;
double ti = wr * recv_imag + wi * recv_real;

// 根据flag选择正确的更新方式
if (flag == 1) {
    local_data[2 * k] = local_real + tr;
    local_data[2 * k + 1] = local_imag + ti;
} else {
    local_data[2 * k] = local_real - tr;
    local_data[2 * k + 1] = local_imag - ti;
}
}
delete[] recv_buf;

```

```

delete[] recv_buf;
} else {
    // 进程内蝶形运算
    for (int k = 0; k < local_N; k += group_size) { //遍历蝶形运算组, local_N/2^(stage+1)
        for (int j = 0; j < stride; ++j) { //每个蝶形运算组内蝶形运算
            int idx = k + j;

            double wr = W_N[2 * j * (N/group_size)];
            double wi = W_N[2 * j * (N/group_size) + 1];
            // 两个计算数的下标
            int even_idx = 2 * (idx);
            int odd_idx = 2 * (idx + stride);

            double tr = wr * local_data[odd_idx] - wi * local_data[odd_idx + 1];
            double ti = wr * local_data[odd_idx + 1] + wi * local_data[odd_idx];

            local_data[odd_idx] = local_data[even_idx] - tr;
            local_data[odd_idx + 1] = local_data[even_idx + 1] - ti;
            local_data[even_idx] += tr;
            local_data[even_idx + 1] += ti;
        }
    }
}
}
}
}

```

运行代码结果如下：

```

● cake@CAK:~/并行程序设计/hw8$ mpirun -np 2 ./fft_mpi_v2 2 10000

run 10000 times FFT
Total time for 10000 FFT+IFFT iterations: 0.0195166 seconds
Average time per FFT+IFFT: 1.95166e-06 seconds

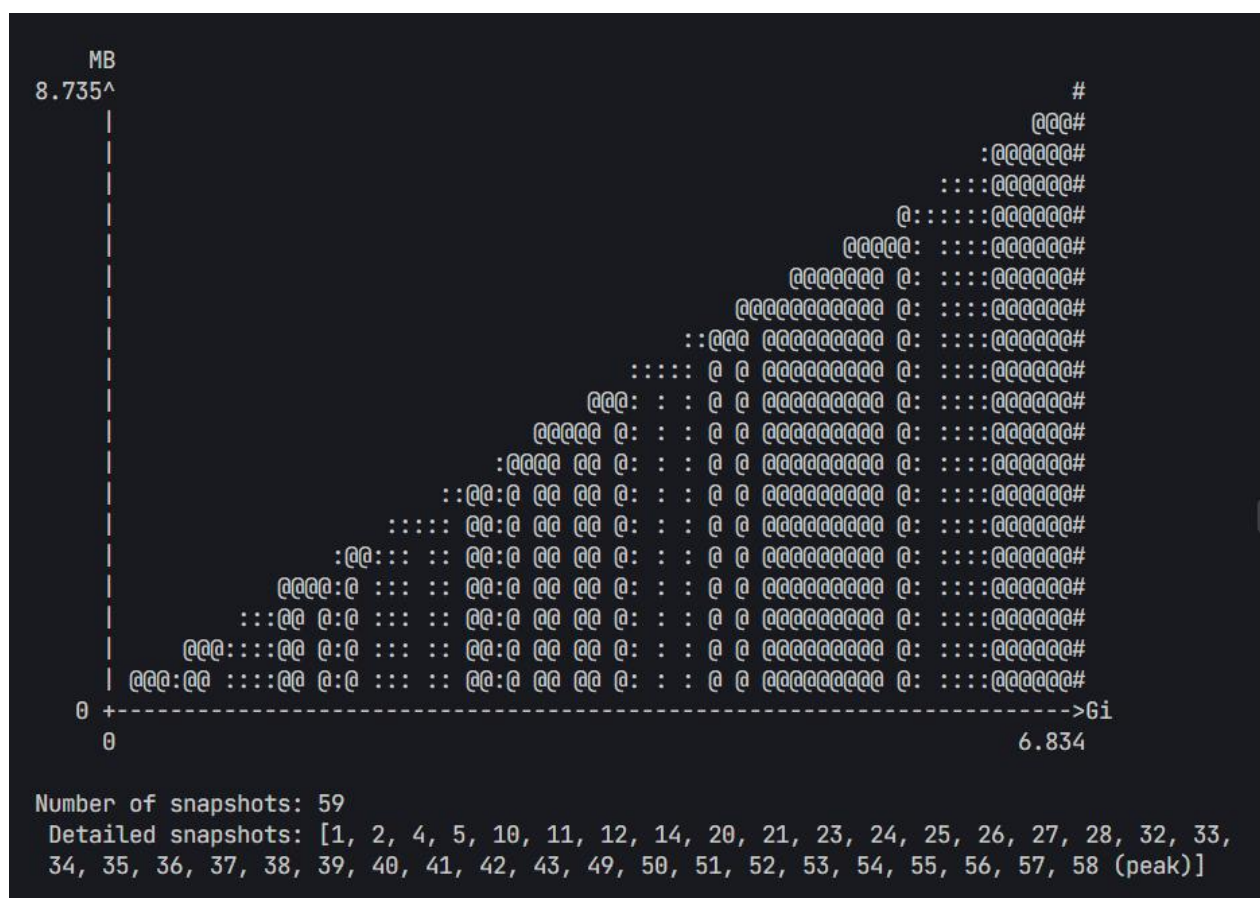
```

任务 2

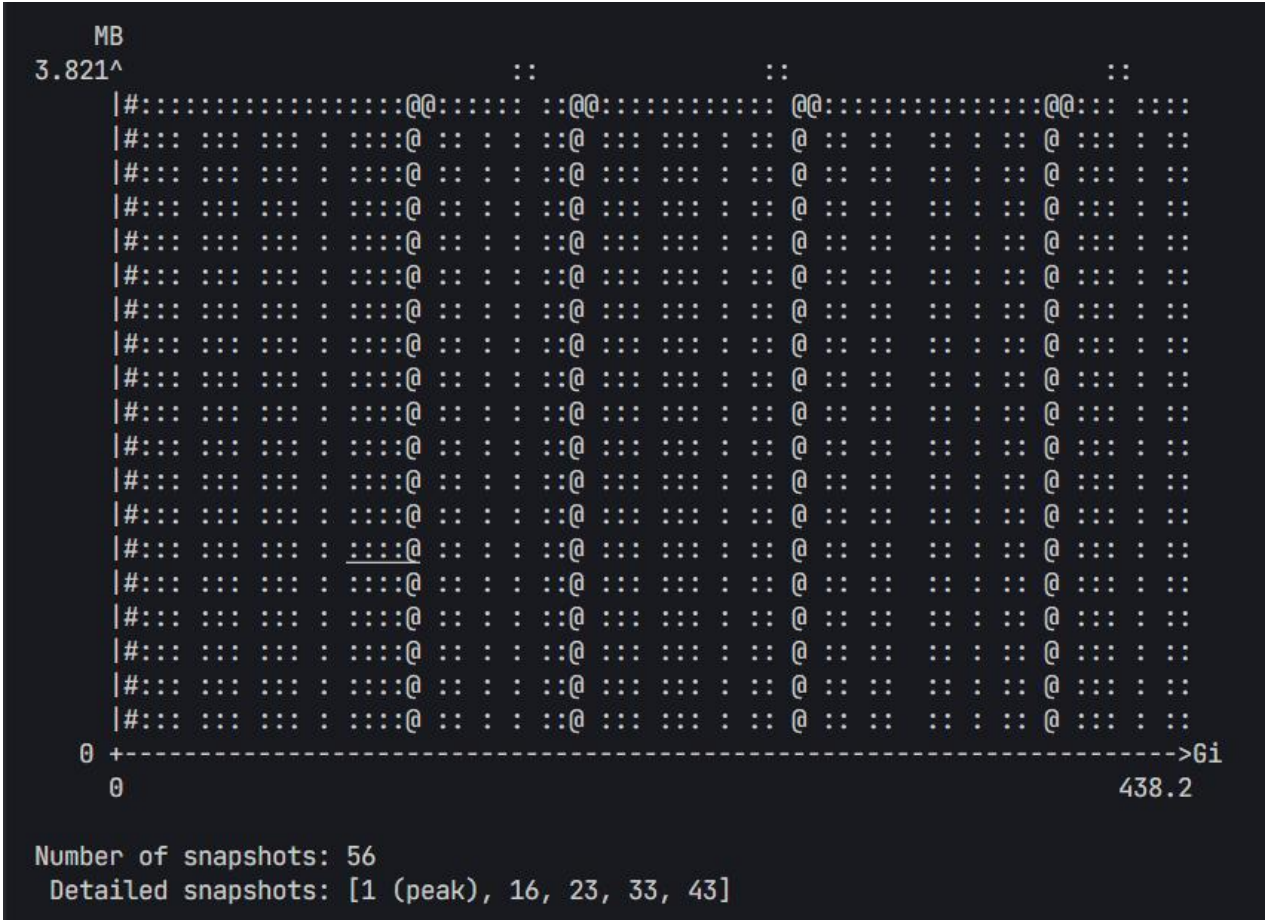
对任务 6 进行修改，使其可以从外部接收参数设置问题规模和线程大小。由于二维数组不能通过外部参数设置规模，所以这里我不再使用二维数组。这样会导致一定的性能损失，不过还在可接受范围内。

使用 Valgrind massif 工具集来完成对 Lab6 实现的 `parallel_for` 版本 `heated_plate_openmp` 应用的分析。Valgrind 是一个用于 Linux 系统的动态分析工具，它可以帮助开发者检测和调试程序中的多种问题，包括但不限于内存管理问题、性能分析、死锁检测等。Valgrind 通过模拟 CPU 指令来运行程序，从而能够在运行时跟踪和分析程序的行为。

运行命令 `valgrind --tool=massif --stacks=yes ./diy`。它将收集程序运行栈内内存消耗的信息，并生成报告。运行后输出报告的内容如下：



作为参照，收集 `parallel_for` 的内存分析图如下：



3. 实验结果

任务 1

作为对比，串行版本运行结果如下：

```
FFT_SERIAL
C++ version

Demonstrate an implementation of the Fast Fourier Transform
of a complex data vector.

Accuracy check:

FFT ( FFT ( X(1:N) ) ) == N * X(1:N)

      N      NITS      Error      Time      Time/Call      MFLOPS

      2      10000      7.85908e-17      0.000372      1.86e-08      537.634
      4      10000      1.20984e-16      0.001264      6.32e-08      632.911
      8      10000      6.8208e-17      0.002515      1.2575e-07      954.274
     16      10000      1.43867e-16      0.009161      4.5805e-07      698.614
     32      1000      1.33121e-16      0.002574      1.287e-06      621.601
     64      1000      1.77654e-16      0.004627      2.3135e-06      829.911
    128      1000      1.92904e-16      0.008841      4.4205e-06      1013.46
    256      1000      2.09232e-16      0.017012      8.506e-06      1203.86
    512      100      1.92749e-16      0.003183      1.5915e-05      1447.69
   1024      100      2.31209e-16      0.007776      3.888e-05      1316.87
   2048      100      2.44501e-16      0.021742      0.00010871      1036.15
   4096      100      2.47659e-16      0.039609      0.000198045      1240.93
   8192      10      2.57125e-16      0.007011      0.00035055      1518.98
  16384      10      2.7363e-16      0.021525      0.00107625      1065.63
  32768      10      2.92413e-16      0.038336      0.0019168      1282.14
  65536      10      2.83355e-16      0.07504      0.003752      1397.36
 131072      1      3.14231e-16      0.015081      0.0075405      1477.5
 262144      1      3.216e-16      0.038819      0.0194095      1215.54
 524288      1      3.28266e-16      0.074695      0.0373475      1333.62
1048576      1      3.28448e-16      0.159383      0.0796915      1315.79

FFT_SERIAL:
Normal end of execution.

18 May 2025 07:52:13 PM
```

收集运行结果，如下表，除了第一个进程外，其余进程全部用 4 进程来运行

长度	迭代	运行时间
2	10000	0.0195166
4	10000	0.0460948
8	10000	0.0590378

16	10000	0.0465865
32	1000	0.00483093
64	1000	0.00647243
128	1000	0.0166719
256	1000	0.0260449
512	100	0.00346424
1024	100	0.033508
2048	100	0.0252143
4096	100	0.0434084
8192	10	0.00866215
16384	10	0.0229234
32768	10	0.0313854
65536	10	0.0707803
131072	1	0.0154523
262144	1	0.0322826
524288	1	0.0645524
1048576	1	0.15386

通过与串行版本的对比可以发现，其实相差都不太大。甚至在数据量较小时性能差于串行版本。分析还是问题规模不够大（运行的时间在一秒之内），进程的创建以及通信开销还是占据了一定程度的性能损耗。

将数据规模加大，结果如下：

```
cake@CAK:~/并行程序设计/hw8$ mpirun -np 1 ./fft_mpi_v2 1048576 50
run 50 times FFT
Total time for 50 FFT+IFFT iterations: 13.7883 seconds
Average time per FFT+IFFT: 0.275766 seconds
cake@CAK:~/并行程序设计/hw8$ mpirun -np 4 ./fft_mpi_v2 1048576 50
run 50 times FFT
Total time for 50 FFT+IFFT iterations: 7.72541 seconds
Average time per FFT+IFFT: 0.154508 seconds
cake@CAK:~/并行程序设计/hw8$ mpirun -np 2 ./fft_mpi_v2 1048576 50
run 50 times FFT
Total time for 50 FFT+IFFT iterations: 10.1241 seconds
Average time per FFT+IFFT: 0.202483 seconds
```

可以看到进程数增加可以提升性能，但并不是与进程呈正相关，分析这还是因为通信的开销的影响。

任务 2

首先完成任务 6 任务规模和线程数目对问题的影响的分析，列出表格如下：

线程数	任务规模				
	128	256	512	1024	
1	1.056125	7.615765	38.236703	154.228904	
2	1.341127	6.245502	25.344006	82.814727	
4	1.786945	6.145412	16.695787	53.286074	
8	3.076361	8.344881	20.996652	50.245264	

对比结果可以发现，线程开销占据了开销的很大比重，在数据规模较大时可以通过线程数的增加提升性能，但是当线程数增加到一定的值后，线程开销过大导致性能降低，或者增加不明显。

使用 Valgrind massif 工具集获取的性能数据如上图，不管什么规模，数据的趋势都和上图一样，可以发现，自定义实现的 `parallel_for` 相比于 `parallel_for` 内存使用不够充分，并没有挖掘硬件本身的全部性能，而官方实现的 `parallel_for` 基本上全程拉满，内存利用更为密集。并且再次观察可以明显发现，`parallel_for` 的内存分配和释放更加规律，而自身实现的代码内存使用明显比较混乱，并且分配更多，这是因为大量的线程创建导致的，而官方 `parallel_for` 的线程创建少得多，这也是另一个自定义 `parallel_for` 性能不如官方的原因。

4. 实验感想

通过本次实验，我了解了快速傅里叶变换的并行化方法，并尝试了自己的实践。同时我比较了 `parallel_for` 的性能表现，这在之前的实验中也有提及，这次我发现官方的 `parallel_for` 的内存管理相比与自定义的代码来说好太多了，并且建立的线程也少很多，这也是性能高的原因之一。这次实验花费的精力真是不小，尤其是被快速傅里叶变换的各种数据传递和该选择哪一个旋转因子搞得头痛。不过最后的效果还是相比我自己来说有进步。这次真的受益匪浅。